There are 210 lines of code being used in the VSC file “cryptography_ui.py”, and 123 lines of actual written code while the rest are either blank lines or mark the sections divided into their common tasks:

- Sieve of Eratosthenes: which was built to prune the first 1000 primes
- Miller-Rabin Primality Test: to check if a chosen number is prime or not
- Prime Generator with Sieve Pruning: checking first with trial division if a number is prime followed by the Miller-Rabin test if probabilistically a number is prime
- Extended Euclidean + Modular Inverse: modular inverse is used for computing the private key exponent; the extended Euclidean algorithm is needed for ensuring that the greatest common divisor (gcd) of $\big(e, \varphi(n)\big) = 1$ (necessary for RSA encryption)
- RSA Utility Functions: the power_mod function is used for the encryption and decryption of the messages; the next function is the public key exponent; the following functions are for converting strings into integers and encodes them as bytes and then decodes those bytes into integers and converts them back to the original message
- Function Test Suite: backend operations where each function is tested to ensure that the algorithms work correctly
- RSA Key Generation + Enryption + Decryption: bits are set to 1024, variables are assigned their functions and keys are generated to be used later on in the UI
    - Encryption: user puts in the message they wish to encrypt; if it exceeds the modulus, the algorithm raises error; the public key component is to be copied and put in to encrypt the text ($message^e \ mod \ n$) when prompted and if the user puts in the wrong component, the function raises an error
    - Decryption: same idea as above, but instead of putting in messages, they are prompted to put in the private key component; if entered incorrectly, the function raises an error; otherwise, the algorithm goes ahead and decrypts the message ($cipher^d \ mod \ n$)
- Output: the function shows both the ciphertext as well as the decrypted original message

I tested the code several times with as diverse versions of messages as I could think of using every combination of the 26 letters in the English alphabet, characters such as ä, ö, å as well as digits from 0 to 9 and special characters and they all worked out just fine every time.

The document “unittesting.py” is the document which has the unit testing. To get the best results and to ensure that the built algorithm “cryptography_ui.py” works correctly, the unit tests should be tested with large prime numbers that match what is usually used in real life RSA functions for encrypting and decrypting messages. My unit tests have both small and large numbers which all passed the tests just fine. And then I also added large prime numbers to test them out as I was instructed to do to check if my algorithm worked successfully or not. The testing can be reproduced with small and or large numbers different than the ones I used, although the larger the input the better it is. Although sometimes it might take longer to perform the operations since computing the modular inverse; prime numbers et cetera would take longer.

There were 451 lines of code in the VSC file, give or take a few breaks and comments inline. The actual number of lines of code that are used in the unit tests are 322. The unittesting.py document used eight out of the twelve given functions in cryptographyui.py document. Thus, the function coverage is 66.67 out of the total functions in the RSA algorithm I built. These functions include:

- The Miller-Rabin test
- euclid_gcd
- extended_gcd
- mod_inverse
- power_mod
- int_to_string
- string_to_int

The line coverage was about 50.41% since the unittesting.py used 62 lines of code (the functions used in the unit tests) out of the 123 lines of code in the cryptography_ui.py document. As for branch coverage---- the number of “if” and “if/else” statements tested---- it was 58.33% since unittesting.py document used seven out of twelve “if” and “if/else” statements written in the cryptography_ui.py file.

It’s for the best to do tests repeatedly to ensure the main idea works. The public key component “e” is constantly the same; “n” and “d” are different every time---- the former a part of both the public and private keys and the latter a part of only the private key. The programme itself raises value error if the values (e and d) entered are incorrect and thus the user would have to start over.